Exercise 13.1

The placement of the text is always centered vertically.

Exercise 13.2

The parameter is used as the title for the frame.

Exercise 13.3

The code to create a button:

```
   JButton button = new JButton("I am a button. I can display some
text. You can also click me.");
   contentPane.add(button);
```

Exercise 13.4

The code to create another button:

```
   JButton anotherButton = new JButton("I am also a button.");
   contentPane.add(anotherButton);
```

Only the last button is visible. It looks like it replaces the first button that was added.

Exercise 13.5

Nothing happens when selecting a menu item. This is because no event handler has been added to the menu item yet.

Exercise 13.6

The method to create the menu bar now looks like this:

```
    private void makeMenuBar()
    {
        JMenuBar menubar = new JMenuBar();
        frame.setJMenuBar(menubar);

        JMenu fileMenu = new JMenu("File");
        menubar.add(fileMenu);

        JMenuItem openItem = new JMenuItem("Open");
        fileMenu.add(openItem);

        JMenuItem quitItem = new JMenuItem("Quit");
        fileMenu.add(quitItem);

        JMenu helpMenu = new JMenu("Help");
        menubar.add(helpMenu);
```

```
        JMenuItem about = new JMenuItem("About ImageViewer");
        helpMenu.add(about);
    }
```

Exercise 13.15

When resizing after an image is loaded it does not resize the image, but only the frame.

Resizing and then opening an image does not have any effect because the loading an image resizes the frame to fit the new image.

Exercise 13.16

The panel now only shows the label with the version number. It does not show any image.

Exercise 13.17

If there is no `CENTER` component then the other areas occupy its space if the frame is small, but the area remains empty once the other components have enough space to display their contents in full.

Exercise 13.18

The calculator project is using `BorderLayout` and `GridLayout`.

Exercise 13.19

The BlueJ editor window is made using `BorderLayout` and `BoxLayout`. `BorderLayout` is used for the main panel, the toolbar is using `BoxLayout`, and the status panel is using `BorderLayout`.

Exercise 13.20

The *Use Library Class* dialog is using `BorderLayout, BoxLayout` and `FlowLayout`.

Exercise 13.23

Solutions to exercises which contain programming can be found in the projects on the CD (`imageviewer1-0`, `imageviewer2-0`, etc.)

Exercise 13.24

The `frame.repaint()` method updates the display of the image after we have modified the contents of the image.

Exercise 13.26

It displays the following message in the status label: `No image loaded`.

Exercise 13.27

The `darker()` method gets the width and height of the image, and then goes through each pixel and changes the color of the pixels, applying the `darker()` method belonging to the class `java.awt.Color`.

Exercise 13.28

See `imageviewer1-0`.

Exercise 13.29

See `imageviewer1-0`.

Exercise 13.31

See `imageviewer1-0`.

Exercise 13.32

`static void showMessageDialog(Component parentComponent, Object message)`
Brings up an information-message dialog titled "Message".

`static void showMessageDialog(Component parentComponent, Object message, String title, int messageType)`
Brings up a dialog that displays a message using a default icon determined by the messageType parameter.

`static void showMessageDialog(Component parentComponent, Object message, String title, int messageType, Icon icon)`
Brings up a dialog displaying a message, specifying all parameters.

We should use the second method as this allows us to specify a proper title for the dialog.

Exercise 13.33

See `imageviewer1-0`.

Exercise 13.34

An `ActionListener` is notified when the *Enter* key is pressed.

The user can be prevented from typing in the text field via the method `setEditable(boolean b)`

Text entered into a `JTextField` is stored in a `Document`. A `DocumentListener` can be added to the `Document` in order to receive notification of changes to it.

Exercise 13.36

Once the new filter has been implemented as a subclass of `Filter`, little needs to be added to the `ImageViewer` class. An instance of the new class needs to be added to the list of filters in the `createFilters()` method. The rest is handled via polymorphism.

Exercise 13.37

See the `13-37-imageviewer` project.

Exercise 13.38-13.43

See the `imageviewer-final` project.

Exercise 13.45-13.50

See the `imageviewer3-0` project.

Exercise 13.54

The URL is: http://download.oracle.com/javase/tutorial/uiswing/

Exercise 13.55

A selection of the available layout managers might be:

`BorderLayout`, `BoxLayout`, `CardLayout`, `FlowLayout`, `GridBagLayout`, `GridLayout`, `SpringLayout` and `OverlayLayout`.

The `CardLayout` allows sharing of limited space between alternative views. See `JTabbedPane` for an alternative way to achieve this.

The `GroupLayout` allows separate specification of the horizontal and vertical juxtaposition of components. In effect, components are always arranged relative to each other, but separately in the two dimensions.

Exercise 13.56

See:
`http://download.oracle.com/javase/tutorial/uiswing/components/slider.html`

Exercise 13.57

See:
`http://download.oracle.com/docs/books/tutorial/uiswing/components/tabbedpane.html`

Exercise 13.58

See:
http://download.oracle.com/j2se/1.5.0/docs/api/javax/swing/JSpinner.html

Exercise 13.59

See:
http://download.oracle.com/docs/books/tutorial/uiswing/components/progress.html

Exercise 13.60

Since, in practice, it will be impossible to apply an inverse of every transformation, the simplest approach is simply to retain a copy of the image immediately prior to transformation. Whether it should be possible to apply *undo* multiple times – possibly up to some limit – will depend upon image size and available memory resources.

Exercise 13.72

The existing `infoLabel` component and `showInfo()` method can be used for this. E.g., add the following in the `play()` method of `MusicPlayerGUI`:

```
      Track tk = trackList.get(index);
      showInfo("Now playing: " + tk.getTitle() + " by " +
tk.getArtist());
```

`infoLabel` should be cleared when playing is stopped.

Exercise 13.73

See the `13-73-musicplayer` project.

Exercise 13.75

The following method sets up the `JSlider`.

```
    /**
     * Set up the labels on the slider, based on the length of
     * the track being played.
     * @param trackLength The length of the track (in arbitrary
units).
     */
    private void setSliderLabels(int trackLength)
    {
        if(trackLength > 0) {
            // How many ticks we want.
            int numberOfTicks = 10;
            // Place ticks to the nearest 1000.
            int roundedLength = trackLength - (trackLength %
1000);
            if(roundedLength == 0) {
                roundedLength = trackLength;
            }
```

```
            slider.setMajorTickSpacing(roundedLength /
numberOfTicks);
            slider.setMinorTickSpacing(0);
            slider.setPaintLabels(true);
            slider.setMaximum(trackLength);
        }
    }
```

This method can be called from the `play()` method in `MusicPlayerGUI`:

```
      player.startPlaying(tk.getFilename());
      setSliderLabels(player.getLength());
```

Note that the track must be started playing before its length it obtained.

Exercise 13.76

Adding a `MouseListener` (`MouseAdaptor`) to the `JList` and checking the click count in a callback to `mouseClicked()` is required.

```
      fileList.addMouseListener(new MouseAdapter() {
             public void mouseClicked(MouseEvent e) {
                 if (e.getClickCount() == 2) {
                     play();
                  }
             }
      });
```

Exercise 13.78

A `JTable` provides alignment control with headings.

Exercise 13.79

See the project `13-79-musicplayer`. A Thread is used to repeatedly check whether anything is being played, and to find out the current position, if so.

Exercise 13.80

You can use the `zuul-for-images` project as a starting point for your students, if you wish. This has separated some of the user interface issues from the game control. The project folder contains an `images` folder with some sample images, if required.

A solution is available in the `13-80-zuul-with-images` project.